```
 1  using System;
 2  using System.Collections.Generic;
 3  using System.Linq;
 4  using System.Text;
 5  using System.Threading.Tasks;
 6  using CustomProjectRPG.Entities;
 7  using CustomProjectRPG.Interfaces;
 8  using CustomProjectRPG.Manager;
 9  using CustomProjectRPG.Core.Utilities;
10
11
12
13  namespace CustomProjectRPG.World
14  {
15      public class Area
16      {
17          private readonly string _name;
18          private readonly Vector2D _size; // Area dimensions (width,
              height)
19          private readonly List<Entity> _entities;
20          private readonly List<ICollidable> _staticAssets;
21          private readonly Dictionary<string, Area> _subAreas;
22          private readonly CollisionManager _collisionManager;
23          private bool _isLocked;
24          private string _unlockCondition; // E.g., knowledge clue ID
25
26          public string Name => _name;
27          public Vector2D Size => _size;
28          public IReadOnlyList<Entity> Entities => _entities.AsReadOnly();
29          public IReadOnlyList<ICollidable> StaticAssets =>
              _staticAssets.AsReadOnly();
30          public IReadOnlyDictionary<string, Area> SubAreas =>
              _subAreas.AsReadOnly();
31          public bool IsLocked => _isLocked;
32
33          public Area(string name, Vector2D size, CollisionManager
              collisionManager)
34          {
35              _name = name;
36              _size = size;
37              _entities = new List<Entity>();
38              _staticAssets = new List<ICollidable>();
39              _subAreas = new Dictionary<string, Area>();
40              _collisionManager = collisionManager;
41              _isLocked = false;
42              _unlockCondition = null;
43          }
44
45          public Area(string name, Vector2D size, CollisionManager
```

```
            collisionManager, string unlockCondition)
46              : this(name, size, collisionManager)
47          {
48              _isLocked = true;
49              _unlockCondition = unlockCondition;
50          }
51
52          public void AddEntity(Entity entity)
53          {
54              if (entity == null)
55              {
56                  throw new System.ArgumentNullException(nameof(entity));
57              }
58              if (!_entities.Contains(entity))
59              {
60                  _entities.Add(entity);
61                  _collisionManager.AddCollidable(entity);
62              }
63          }
64
65          public void RemoveEntity(Entity entity)
66          {
67              if (entity == null)
68              {
69                  throw new System.ArgumentNullException(nameof(entity));
70              }
71              if (_entities.Remove(entity))
72              {
73                  _collisionManager.RemoveCollidable(entity);
74              }
75          }
76
77          public void AddStaticAsset(ICollidable asset)
78          {
79              if (asset == null)
80              {
81                  throw new System.ArgumentNullException(nameof(asset));
82              }
83              if (!_staticAssets.Contains(asset))
84              {
85                  _staticAssets.Add(asset);
86                  _collisionManager.AddCollidable(asset);
87              }
88          }
89
90          public void RemoveStaticAsset(ICollidable asset)
91          {
92              if (asset == null)
93              {
```

```
94                  throw new System.ArgumentNullException(nameof(asset));
95              }
96              if (_staticAssets.Remove(asset))
97              {
98                  _collisionManager.RemoveCollidable(asset);
99              }
100         }
101
102         public void AddSubArea(string subAreaName, Area subArea)
103         {
104             if (subArea == null)
105             {
106                 throw new System.ArgumentNullException(nameof(subArea));
107             }
108             if (!_subAreas.ContainsKey(subAreaName))
109             {
110                 _subAreas.Add(subAreaName, subArea);
111             }
112         }
113
114         public void RemoveSubArea(string subAreaName)
115         {
116             _subAreas.Remove(subAreaName);
117         }
118
119         public bool TryUnlock(string knowledgeClue)
120         {
121             if (_isLocked && _unlockCondition == knowledgeClue)
122             {
123                 _isLocked = false;
124                 return true;
125             }
126             return false;
127         }
128
129         public void Update()
130         {
131             foreach (var entity in _entities)
132             {
133                 entity.Update();
134             }
135             _collisionManager.CheckCollisions();
136         }
137
138         public void Draw()
139         {
140             foreach (var entity in _entities)
141             {
142                 entity.Draw();
```

```
143             }
144             // Draw static assets (placeholder for SplashKit rendering)
145             foreach (var asset in _staticAssets)
146             {
147                 if (asset is StaticAsset staticAsset)
148                 {
149                     staticAsset.Draw();
150                 }
151             }
152         }
153 
154         public bool IsWithinBounds(Vector2D position)
155         {
156             return position.X >= 0 && position.X < _size.X && position.Y
                  >= 0 && position.Y < _size.Y;
157         }
158     }
159 }
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Text.Json;
using System.Text.Json.Serialization;
using CustomProjectRPG.Interfaces;
using CustomProjectRPG.Entities.Enum;
using CustomProjectRPG.Entities.Component;
using CustomProjectRPG.Core.Utilities;

namespace CustomProjectRPG.Entities
{



public class BeastEntity : ISerializeJSON<BeastEntity>
    {
        private CustomProjectRPG.Entities.Enum.Enum.BeastType _beastType;
        private CorruptionComponent _corruption;
        private StatsComponent _stats;
        private bool _isAggressive;

        public event Action<string,
          CustomProjectRPG.Entities.Enum.Enum.CorruptionState>
          CorruptionStateChanged; // Event for state updates

        /// <summary>
        /// Initializes a new BeastEntity with the specified beast type
          and initial stats.
        /// </summary>
        public BeastEntity(CustomProjectRPG.Entities.Enum.Enum.BeastType
          beastType)
        {
            _beastType = beastType;
            _corruption = new CorruptionComponent
              (CustomProjectRPG.Entities.Enum.Enum.EntityType.Beast,
              GetInitialCorruption(beastType));
            _stats = new StatsComponent
              (CustomProjectRPG.Entities.Enum.Enum.EntityType.Beast,
              GetInitialHP(), GetInitialATK(), GetInitialDEF(),
              GetInitialAgility());
            _isAggressive = false;
            UpdateAggressiveness();
        }

        /// <summary>
        /// Gets the initial corruption percentage based on beast type.
```

```
41          /// </summary>
42          private float GetInitialCorruption
              (CustomProjectRPG.Entities.Enum.Enum.BeastType beastType)
43          {
44              return beastType switch
45              {
46                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Weak => 0f,
47                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Elite =>
                      50f,
48                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Boss => 75f,
49                  _ => 0f
50              };
51          }
52
53          /// <summary>
54          /// Gets the initial HP based on beast type.
55          /// </summary>
56          private float GetInitialHP()
57          {
58              return _beastType switch
59              {
60                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Weak => 50f,
61                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Elite =>
                      150f,
62                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Boss =>
                      300f,
63                  _ => 50f
64              };
65          }
66
67          /// <summary>
68          /// Gets the initial ATK based on beast type.
69          /// </summary>
70          private float GetInitialATK()
71          {
72              return _beastType switch
73              {
74                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Weak => 5f,
75                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Elite =>
                      15f,
76                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Boss => 25f,
77                  _ => 5f
78              };
79          }
80
81          /// <summary>
82          /// Gets the initial DEF based on beast type.
83          /// </summary>
84          private float GetInitialDEF()
```

```
85          {
86              return _beastType switch
87              {
88                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Weak => 2f,
89                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Elite => 8f,
90                  CustomProjectRPG.Entities.Enum.Enum.BeastType.Boss => 15f,
91                  _ => 2f
92              };
93          }
94
95          /// <summary>
96          /// Gets the initial Agility based on beast type.
97          /// </summary>
98          private float GetInitialAgility()
99          {
100             return _beastType switch
101             {
102                 CustomProjectRPG.Entities.Enum.Enum.BeastType.Weak => 3f,
103                 CustomProjectRPG.Entities.Enum.Enum.BeastType.Elite =>
                      10f,
104                 CustomProjectRPG.Entities.Enum.Enum.BeastType.Boss => 7f,
105                 _ => 3f
106             };
107         }
108
109         /// <summary>
110         /// Updates the beast's aggressiveness based on corruption state.
111         /// </summary>
112         private void UpdateAggressiveness()
113         {
114             var state = _corruption.GetCorruptionState();
115             _isAggressive = state ==
                  CustomProjectRPG.Entities.Enum.Enum.CorruptionState.Full;
116             CorruptionStateChanged?.Invoke($"Beast_{_beastType}",
                  state); // Notify state change
117         }
118
119         /// <summary>
120         /// Applies damage to the beast, updating stats and checking
              death.
121         /// </summary>
122         public void TakeDamage(float amount)
123         {
124             _stats.TakeDamage(amount);
125             if (_stats.HP <= 0)
126             {
127                 // Handle death logic (e.g., drop items, quest update)
128                 Console.WriteLine($"Beast {_beastType} defeated!");
129             }
```

```
130         }
131
132         /// <summary>
133         /// Increases corruption, triggering state and aggressiveness updates.
134         /// </summary>
135         public void IncreaseCorruption(float amount)
136         {
137             _corruption.IncreaseCorruption(amount);
138             UpdateAggressiveness();
139         }
140
141         /// <summary>
142         /// Serializes the beast's state to JSON.
143         /// </summary>
144         public string Serialize()
145         {
146             var data = new BeastData
147             {
148                 BeastType = _beastType,
149                 CorruptionState = _corruption.Serialize(),
150                 StatsState = _stats.Serialize(),
151                 IsAggressive = _isAggressive
152             };
153             return JsonSerializer.Serialize(data);
154         }
155
156         /// <summary>
157         /// Deserializes the beast's state from JSON.
158         /// </summary>
159         public void Deserialize(string json)
160         {
161             var data = JsonSerializer.Deserialize<BeastData>(json)
162                 ?? throw new JsonException("Invalid JSON format for beast state.");
163             _beastType = data.BeastType;
164             _corruption.Deserialize(data.CorruptionState);
165             _stats.Deserialize(data.StatsState);
166             _isAggressive = data.IsAggressive;
167             UpdateAggressiveness();
168         }
169
170         private class BeastData
171         {
172             [JsonPropertyName("beastType")]
173             public CustomProjectRPG.Entities.Enum.Enum.BeastType BeastType { get; set; }
174
175             [JsonPropertyName("corruptionState")]
```

```
176             public string CorruptionState { get; set; }
177
178             [JsonPropertyName("statsState")]
179             public string StatsState { get; set; }
180
181             [JsonPropertyName("isAggressive")]
182             public bool IsAggressive { get; set; }
183         }
184     }
185 }
186
187
```

```
 1  using CustomProjectRPG.Entities;
 2  using CustomProjectRPG.Interfaces;
 3  using System;
 4  using System.Collections.Generic;
 5  using System.Linq;
 6  using System.Text;
 7  using System.Threading.Tasks;
 8
 9  namespace CustomProjectRPG.Manager
10  {
11      public class CollisionManager
12      {
13          private readonly List<ICollidable> _collidables;
14
15          public CollisionManager()
16          {
17              _collidables = new List<ICollidable>();
18          }
19
20          public void AddCollidable(ICollidable collidable)
21          {
22              if (collidable == null)
23              {
24                  throw new System.ArgumentNullException(nameof(collidable));
25              }
26              if (!_collidables.Contains(collidable))
27              {
28                  _collidables.Add(collidable);
29              }
30          }
31
32          public void RemoveCollidable(ICollidable collidable)
33          {
34              if (collidable == null)
35              {
36                  throw new System.ArgumentNullException(nameof(collidable));
37              }
38              _collidables.Remove(collidable);
39          }
40
41          public void CheckCollisions()
42          {
43              // Use a nested loop to check pairwise collisions
44              for (int i = 0; i < _collidables.Count; i++)
45              {
46                  for (int j = i + 1; j < _collidables.Count; j++)
47                  {
48                      ICollidable first = _collidables[i];
49                      ICollidable second = _collidables[j];
```

```
50
51                      if (first.Hitbox.Intersects(second.Hitbox))
52                      {
53                          // Notify both entities of the collision
54                          if (first is Entity entity1)
55                          {
56                              entity1.OnCollision(second);
57                          }
58                          if (second is Entity entity2)
59                          {
60                              entity2.OnCollision(first);
61                          }
62                      }
63                  }
64              }
65          }
66
67          public void Clear()
68          {
69              _collidables.Clear();
70          }
71      }
72  }
73
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using CustomProjectRPG.Entities.Enum;
7  using Enum = CustomProjectRPG.Entities.Enum.Enum;
8
9
10 namespace CustomProjectRPG.Entities.Component
11 {
12
13
14     public static class ComponentFactory
15     {
16         public static object CreateComponent
             (CustomProjectRPG.Entities.Enum.Enum.EntityType entityType)
17         {
18             return entityType switch
19             {
20                 CustomProjectRPG.Entities.Enum.Enum.EntityType.Player =>
                    new CorruptionComponent(entityType),
21                 CustomProjectRPG.Entities.Enum.Enum.EntityType.NPC => new
                    CorruptionComponent(entityType),
22                 CustomProjectRPG.Entities.Enum.Enum.EntityType.Beast =>
                    new CorruptionComponent(entityType),
23                 _ => throw new ArgumentException("Unsupported entity
                    type.", nameof(entityType))
24             };
25         }
26
27         // Overload for specific component types if needed
28         public static T CreateSpecificComponent<T>
             (CustomProjectRPG.Entities.Enum.Enum.EntityType entityType)
             where T : class
29         {
30             if (typeof(T) == typeof(CorruptionComponent))
31             {
32                 return new CorruptionComponent(entityType) as T ?? throw
                    new InvalidOperationException("Failed to create
                    component.");
33             }
34             if (typeof(T) == typeof(StatsComponent))
35             {
36                 return new StatsComponent(entityType) as T ?? throw new
                    InvalidOperationException("Failed to create component.");
37             }
38             throw new ArgumentException("Unsupported component type.",
                nameof(T));
```

```
39          }
40      }
41  }
42
```

```
 1  using CustomProjectRPG.Core.Utilities;
 2  using CustomProjectRPG.Interfaces;
 3  using System;
 4  using System.Collections.Generic;
 5  using System.Linq;
 6  using System.Text;
 7  using System.Text.Json;
 8  using System.Text.Json.Serialization;
 9  using System.Threading.Tasks;
10  using Enum = CustomProjectRPG.Entities.Enum.Enum;
11
12  namespace CustomProjectRPG
13  {
14
15
16
17      public class CorruptionComponent :
          ISerializeJSON<CorruptionComponent>, ICorruption
18      {
19          private float _corruptionPercentage;
20          private int _warningCount;
21          private Enum.EntityType _entityType;
22
23          public float CorruptionPercentage => _corruptionPercentage;
24          public int WarningCount => _warningCount;
25
26
27          /// Initializes a new CorruptionComponent for a specific entity
              type with an optional initial corruption value.
28
29          public CorruptionComponent(Enum.EntityType entityType, float
              initialCorruption = 0f)
30          {
31              _entityType = entityType;
32              _corruptionPercentage = Math.Clamp(initialCorruption, 0f,
                  100f);
33              if (_entityType == Enum.EntityType.Player)
34              {
35                  _warningCount = 0;
36              }
37          }
38
39
40          /// Increases corruption by the specified amount, capping at 100%.
41
42          public void IncreaseCorruption(float amount)
43          {
44              if (amount < 0) throw new ArgumentException("Amount must be
                    non-negative.", nameof(amount));
```

```
45              _corruptionPercentage = Math.Min(_corruptionPercentage +
                  amount, 100f);
46          }
47
48
49          /// Decreases corruption by the specified amount, flooring at 0%.
50
51          public void DecreaseCorruption(float amount)
52          {
53              if (amount < 0) throw new ArgumentException("Amount must be
                  non-negative.", nameof(amount));
54              _corruptionPercentage = Math.Max(_corruptionPercentage -
                  amount, 0f);
55          }
56
57
58          /// Returns the corruption state (used primarily for beasts).
59
60          public Enum.CorruptionState GetCorruptionState()
61          {
62              if (_corruptionPercentage == 0f) return
                    Enum.CorruptionState.Normal;
63              if (_corruptionPercentage == 100f) return
                    Enum.CorruptionState.Full;
64              return Enum.CorruptionState.Partial;
65          }
66
67
68          /// Checks if the player should receive a corruption warning from
              the daughter.
69
70          public bool ShouldShowWarning()
71          {
72              return _entityType == Enum.EntityType.Player &&
                    _corruptionPercentage >= 77f && _warningCount < 3;
73          }
74
75
76          /// Increments the warning count for the player.
77
78          public void IncrementWarningCount()
79          {
80              if (_entityType == Enum.EntityType.Player)
81              {
82                  _warningCount++;
83              }
84          }
85
86
```

```
87          /// Checks if the player's corruption triggers the bad ending.
88
89          public bool ShouldTriggerBadEnding()
90          {
91              return _entityType == Enum.EntityType.Player &&
                  _corruptionPercentage >= 77f && _warningCount >= 3;
92          }
93
94
95          /// Checks if an NPC's dialogue is corrupted (≥50% corruption).
96
97          public bool IsDialogueCorrupted()
98          {
99              return _entityType == Enum.EntityType.NPC &&
                  _corruptionPercentage >= 50f;
100         }
101
102
103         /// Checks if an NPC's house is a placeholder (≥60% corruption).
104
105         public bool IsHousePlaceholder()
106         {
107             return _entityType == Enum.EntityType.NPC &&
                  _corruptionPercentage >= 60f;
108         }
109
110
111         /// Checks if an NPC stops responding (≥80% corruption).
112
113         public bool IsNoResponse()
114         {
115             return _entityType == Enum.EntityType.NPC &&
                  _corruptionPercentage >= 80f;
116         }
117
118
119         /// Updates corruption at the start of a new loop based on entity
              type.
120
121         public void OnNewLoop()
122         {
123             if (_entityType == Enum.EntityType.Player)
124             {
125                 IncreaseCorruption(10f); // Player: +10% per loop
126             }
127             else if (_entityType == Enum.EntityType.NPC)
128             {
129                 Random rand = new Random();
130                 float increase = rand.Next(11, 18); // NPC: +11% to 17%
```

```
                    per loop
131                 IncreaseCorruption(increase);
132             }
133             // Beasts: No change (set at spawn)
134         }
135
136
137         /// Calculates the stat multiplier for the player based on
              corruption level.
138
139         public float GetStatMultiplier()
140         {
141             if (_entityType != Enum.EntityType.Player) return 1.0f;
142             if (_corruptionPercentage < 50f) return 1.0f;
143             int steps = (int)((_corruptionPercentage - 50f) / 10f);
144             float multiplier = 0.8f - steps * 0.1f; // 20% decrease at
                  50%, then 10% per 10%
145             return Math.Max(multiplier, 0f);
146         }
147
148
149         public event Action<float> CorruptionChanged;
150         private void OnCorruptionChanged() => CorruptionChanged?.Invoke
              (_corruptionPercentage);
151
152         /// Serializes the corruption state to JSON.
153
154         public string Serialize()
155         {
156             var data = new CorruptionData
157             {
158                 CorruptionPercentage = _corruptionPercentage,
159                 WarningCount = _warningCount
160             };
161             return SerializeJSON.ToJson(data);
162         }
163
164
165         /// Deserializes the corruption state from JSON.
166
167         public void Deserialize(string json)
168         {
169             var data = SerializeJSON.FromJson<CorruptionData>(json);
170             _corruptionPercentage = Math.Clamp(data.CorruptionPercentage,
                  0f, 100f);
171             if (_entityType == Enum.EntityType.Player)
172             {
173                 _warningCount = data.WarningCount;
174             }
```

```
175         }
176
177         private class CorruptionData
178         {
179             [JsonPropertyName("corruptionPercentage")]
180             public float CorruptionPercentage { get; set; }
181
182             [JsonPropertyName("warningCount")]
183             public int WarningCount { get; set; }
184         }
185
186     }
187 }
188
```

```
 1  using CustomProjectRPG.Core.Utilities;
 2  using CustomProjectRPG.Interfaces;
 3  using System;
 4  using System.Collections.Generic;
 5  using System.Linq;
 6  using System.Text;
 7  using System.Text.Json;
 8  using System.Text.Json.Serialization;
 9  using System.Threading.Tasks;
10  using static CustomProjectRPG.Entities.Enum.Enum;
11
12  namespace CustomProjectRPG.Dialogue
13  {
14
15      public class DialogueManager : ISerializeJSON<DialogueManager>
16      {
17          private Dictionary<string, Dictionary<string, string>>
              _npcDialogues; // NPC ID -> Dialogue ID -> Text
18          private ICorruption _corruption;
19          private string _currentNpcId;
20          private string _currentDialogueId;
21
22          public event Action<string> DialogueChanged; // Observer pattern
              for UI updates
23
24
25          /// Initializes a new DialogueManager with a corruption reference
              and loads dialogues from JSON.
26
27          public DialogueManager(ICorruption corruption, string
              jsonDialogues = null)
28          {
29              _corruption = corruption ?? throw new ArgumentNullException
                  (nameof(corruption));
30              _npcDialogues = new Dictionary<string, Dictionary<string,
                  string>>();
31              _currentNpcId = string.Empty;
32              _currentDialogueId = string.Empty;
33
34              if (!string.IsNullOrEmpty(jsonDialogues))
35              {
36                  LoadDialoguesFromJson(jsonDialogues);
37              }
38          }
39
40
41          /// Loads dialogue data from a JSON string, organized by NPC ID.
42
43          private void LoadDialoguesFromJson(string json)
```

```
44          {
45              var dialogues = JsonSerializer.Deserialize<Dictionary<string,
                  Dictionary<string, string>>>(json)
46                  ?? throw new JsonException("Invalid JSON format for
                      dialogues.");
47              _npcDialogues = dialogues;
48          }
49
50
51          /// Retrieves available dialogue options for the current NPC
                based on corruption state.
52
53          public List<string> GetDialogueOptions()
54          {
55              if (!_npcDialogues.ContainsKey(_currentNpcId) || !
                    _npcDialogues[_currentNpcId].Any())
56              {
57                  return new List<string> { "No dialogue available for this
                        NPC." };
58              }
59
60              var options = new List<string>();
61              var npcDialogues = _npcDialogues[_currentNpcId];
62
63              if (npcDialogues.TryGetValue("normal", out var
                    normalDialogue))
64              {
65                  options.Add(normalDialogue);
66              }
67              if (_corruption.IsDialogueCorrupted() &&
                    npcDialogues.TryGetValue("corrupted", out var
                    corruptedDialogue))
68              {
69                  options.Add(corruptedDialogue);
70              }
71              if (_corruption.IsNoResponse() && npcDialogues.TryGetValue
                    ("noResponse", out var noResponseDialogue))
72              {
73                  options.Clear();
74                  options.Add(noResponseDialogue);
75              }
76
77              return options.Count > 0 ? options : new List<string> { "No
                    valid dialogue options." };
78          }
79
80
81          /// Sets and displays the current dialogue for the specified NPC,
                triggering an event.
```

```
82
83          public void ShowDialogue(string npcId, string dialogueId)
84          {
85              if (!_npcDialogues.ContainsKey(npcId) || !_npcDialogues
                  [npcId].ContainsKey(dialogueId))
86              {
87                  throw new ArgumentException($"Invalid NPC ID or dialogue
                      ID: {npcId}/{dialogueId}.", nameof(dialogueId));
88              }
89              _currentNpcId = npcId;
90              _currentDialogueId = dialogueId;
91              OnDialogueChanged();
92          }
93
94
95          /// Gets the current dialogue text for the active NPC.
96
97          public string GetCurrentDialogue()
98          {
99              if (string.IsNullOrEmpty(_currentNpcId) ||
                    string.IsNullOrEmpty(_currentDialogueId) ||
100                 !_npcDialogues.ContainsKey(_currentNpcId) || !
                      _npcDialogues[_currentNpcId].ContainsKey
                      (_currentDialogueId))
101             {
102                 return "No dialogue selected.";
103             }
104             return _npcDialogues[_currentNpcId][_currentDialogueId];
105         }
106
107
108         /// Serializes the current dialogue state to JSON.
109
110         public string Serialize()
111         {
112             var data = new DialogueData
113             {
114                 CurrentNpcId = _currentNpcId,
115                 CurrentDialogueId = _currentDialogueId
116             };
117             return JsonSerializer.Serialize(data);
118         }
119
120
121         /// Deserializes the dialogue state from JSON.
122
123         public void Deserialize(string json)
124         {
125             var data = JsonSerializer.Deserialize<DialogueData>(json)
```

```
126                     ?? throw new JsonException("Invalid JSON format for dialogue state.");
127                 if (_npcDialogues.ContainsKey(data.CurrentNpcId) && _npcDialogues[data.CurrentNpcId].ContainsKey(data.CurrentDialogueId))
128                 {
129                     _currentNpcId = data.CurrentNpcId;
130                     _currentDialogueId = data.CurrentDialogueId;
131                 }
132             }
133 
134             private void OnDialogueChanged()
135             {
136                 DialogueChanged?.Invoke(GetCurrentDialogue());
137             }
138 
139             private class DialogueData
140             {
141                 [JsonPropertyName("currentNpcId")]
142                 public string CurrentNpcId { get; set; }
143 
144                 [JsonPropertyName("currentDialogueId")]
145                 public string CurrentDialogueId { get; set; }
146             }
147         }
148     }
149 
```

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using CustomProjectRPG.Core.Utilities;
using CustomProjectRPG.Interfaces;
using CustomProjectRPG.Core;


namespace CustomProjectRPG.Entities
{

    public abstract class Entity : ICollidable, IDraw, IUpdate
    {
        // Private fields for encapsulation.
        private string _name;
        private string _description;
        private Vector2D _position;
        private Hitbox _hitbox;

        // Constructor for the Entity class.
        // It initializes the name, description, position, and hitbox.
        protected Entity(string name, string description, Vector2D
          startPosition, Vector2D hitboxSize)
        {
            _name = name;
            _description = description;
            _position = startPosition;
            _hitbox = new Hitbox(startPosition, hitboxSize);
        }

        // Public properties for controlled access to the private fields.
        public string Name
        {
            get { return _name; }
        }

        public string Description
        {
            get { return _description; }
        }

        public Vector2D Position
        {
            get { return _position; }
            set
            {
```

```
49                  _position = value;
50                  _hitbox.Position = value;
51              }
52          }
53
54          public Hitbox Hitbox
55          {
56              get { return _hitbox; }
57          }
58
59          // Abstract methods to be implemented by derived classes.
60          public virtual bool IsCollidingWith(ICollidable other)
61
62          {
63                  return this.Hitbox.Intersects(other.Hitbox);
64          }
65
66          public abstract void OnCollision(ICollidable other);
67          public abstract void Draw();
68          public abstract void Update();
69
70
71          // Moves the entity to a new position.
72
73          public virtual void MoveTo(float x, float y)
74          {
75              this.Position = new Vector2D(x, y);
76          }
77      }
78
79  }
80
```

```
 1 using System;
 2 using System.Collections.Generic;
 3 using System.Linq;
 4 using System.Text;
 5 using System.Threading.Tasks;
 6
 7 namespace CustomProjectRPG.Entities.Enum
 8
 9 {
10     public static class Enum
11     {
12         public enum BeastType { Weak, Elite, Boss }
13         public enum CorruptionState { Normal, Partial, Full }
14         public enum FavorResult { Success, Insufficient, Invalid }
15
16         public enum QuestType { Daily, Story, Favor }
17
18         public enum EntityType { Player, NPC, Beast }
19     }
20 }
21
```

```
using CustomProjectRPG.Interfaces;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Threading.Tasks;


namespace CustomProjectRPG
{

    public class FavorComponent: ISerializeJSON<FavorComponent>
    {
        private Dictionary<string, int> _favorMap;
        private Dictionary<string, int> _hiddenFavorMap;

        public FavorComponent()
        {
            _favorMap = new Dictionary<string, int>();
            _hiddenFavorMap = new Dictionary<string, int>();
        }

        public int GetFavor(string npcId)
        {
            ValidateNpcId(npcId);
            return _favorMap.ContainsKey(npcId) ? _favorMap[npcId] : 0;
        }

        public int GetHiddenFavor(string npcId)
        {
            ValidateNpcId(npcId);
            return _hiddenFavorMap.ContainsKey(npcId) ? _hiddenFavorMap
              [npcId] : 0;
        }

        public void AdjustFavor(string npcId, int amount)
        {
            ValidateNpcId(npcId);
            if (_favorMap.ContainsKey(npcId))
            {
                _favorMap[npcId] += amount;
            }
            else
            {
                _favorMap[npcId] = amount;
            }
        }
```

```
49
50          public void AdjustHiddenFavor(string npcId, int amount)
51          {
52              ValidateNpcId(npcId);
53              if (_hiddenFavorMap.ContainsKey(npcId))
54              {
55                  _hiddenFavorMap[npcId] += amount;
56              }
57              else
58              {
59                  _hiddenFavorMap[npcId] = amount;
60              }
61          }
62
63          public bool CanUseFavor(string npcId, int cost)
64          {
65              ValidateNpcId(npcId);
66              return _favorMap.ContainsKey(npcId) && _favorMap[npcId] >=
                  cost;
67          }
68
69          public bool UseFavor(string npcId, int cost)
70          {
71              if (CanUseFavor(npcId, cost))
72          {
73              _favorMap[npcId] -= cost;
74              return true;
75          }
76          return false;
77          }
78
79          public string Serialize()
80          {
81              var data = new FavorData
82              {
83                  FavorMap = _favorMap,
84                  HiddenFavorMap = _hiddenFavorMap
85              };
86
87              return JsonSerializer.Serialize(data);
88          }
89
90          public void Deserialize(string json)
91          {
92              if (string.IsNullOrWhiteSpace(json)) return;
93
94              var data = JsonSerializer.Deserialize<FavorData>(json);
95              _favorMap = data?.FavorMap ?? new Dictionary<string, int>();
96              _hiddenFavorMap = data?.HiddenFavorMap ?? new
```

```
                Dictionary<string, int>();
        }

        private void ValidateNpcId(string npcId)
        {
            if (string.IsNullOrWhiteSpace(npcId))
            {
                throw new ArgumentException("NPC ID cannot be null or
                  empty.");
            }
        }

        private class FavorData
        {
            [JsonPropertyName("favorMap")]
            public Dictionary<string, int> FavorMap { get; set; }

            [JsonPropertyName("hiddenFavorMap")]
            public Dictionary<string, int> HiddenFavorMap { get; set; }
        }
    }
}

```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace CustomProjectRPG.Core.Utilities
8  {
9      public struct Hitbox
10
11     {
12         public Vector2D Position { get; set; }
13         public Vector2D Size { get; }
14
15         public Hitbox(Vector2D position, Vector2D size)
16         {
17             Position = position;
18             Size = size;
19         }
20
21         public bool Intersects(Hitbox other)
22         {
23             return !(Position.X + Size.X < other.Position.X ||
24                      Position.X > other.Position.X + other.Size.X ||
25                      Position.Y + Size.Y < other.Position.Y ||
26                      Position.Y > other.Position.Y + other.Size.Y);
27         }
28     }
29
30 }
31
32
```

```
 1  using CustomProjectRPG.Core.Utilities;
 2  using System;
 3  using System.Collections.Generic;
 4  using System.Linq;
 5  using System.Text;
 6  using System.Threading.Tasks;
 7
 8  namespace CustomProjectRPG.Interfaces
 9  {
10       public interface ICollidable
11      {
12          Hitbox Hitbox { get; }
13      }
14  }
15
```

```
 1  using System;
 2  using System.Collections.Generic;
 3  using System.Linq;
 4  using System.Text;
 5  using System.Threading.Tasks;
 6  using Enum = CustomProjectRPG.Entities.Enum.Enum;
 7
 8  namespace CustomProjectRPG.Interfaces
 9  {
10      public interface ICorruption
11      {
12          float GetStatMultiplier();
13          Enum.CorruptionState GetCorruptionState();
14
15          bool IsDialogueCorrupted();
16          bool IsNoResponse();
17
18          float CorruptionPercentage { get; }
19
20          void IncreaseCorruption(float amount);
21
22          void DecreaseCorruption(float amount);
23
24          string Serialize();
25
26          void Deserialize(string json);
27
28
29      }
30  }
31
```

```
 1 using System;
 2 using System.Collections.Generic;
 3 using System.Linq;
 4 using System.Text;
 5 using System.Threading.Tasks;
 6
 7 namespace CustomProjectRPG.Core.Utilities
 8 {
 9     public interface IDraw
10     {
11         void Draw();
12     }
13 }
14
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace CustomProjectRPG.Interfaces
8  {
9
10     public interface IQuestManager
11     {
12         void StartQuest(CustomProjectRPG.Entities.Enum.Enum.QuestType
             type, string questId);
13         bool IsQuestComplete(string questId);
14         CustomProjectRPG.Entities.Enum.Enum.FavorResult CheckFavorResult
             (string questId);
15     }
16 }
17
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace CustomProjectRPG.Interfaces
8  {
9      public interface ISerializeJSON <T>
10     {
11
12         string Serialize();
13         void Deserialize(string json);
14
15     }
16 }
17
```

```cs
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CustomProjectRPG.Core.Utilities
{
    interface IUpdate
    {

        void Update();

    }
}

```

```
using CustomProjectRPG.Interfaces;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Threading.Tasks;
using Enum = CustomProjectRPG.Entities.Enum.Enum;

namespace CustomProjectRPG.Manager
{
    public class QuestManager : ISerializeJSON<QuestManager>, IQuestManager
    {
        private Dictionary<string, QuestData> _quests; // Quest ID -> QuestData
        private ICorruption _corruption; // For favor-based quest adjustments

        public event Action<string, bool> QuestUpdated; // Event for quest status changes (questId, isComplete)


        /// Initializes a new QuestManager with a corruption reference.

        public QuestManager(ICorruption corruption)
        {
            _corruption = corruption ?? throw new ArgumentNullException(nameof(corruption));
            _quests = new Dictionary<string, QuestData>();
        }


        /// Starts a new quest with the specified type and ID.

        public void StartQuest(CustomProjectRPG.Entities.Enum.Enum.QuestType type, string questId)
        {
            if (string.IsNullOrEmpty(questId)) throw new ArgumentException("Quest ID cannot be null or empty.", nameof(questId));
            if (_quests.ContainsKey(questId)) throw new ArgumentException("Quest already exists.", nameof(questId));

            _quests[questId] = new QuestData
            {
                Type = type,
                IsActive = true,
```

```
41                  Progress = 0,
42                  RequiredProgress = GetRequiredProgress(type),
43                  IsComplete = false
44              };
45              QuestUpdated?.Invoke(questId, false); // Notify quest start
46          }
47
48
49          /// Updates the progress of a quest.
50
51          public void UpdateQuestProgress(string questId, int amount)
52          {
53              if (!_quests.ContainsKey(questId) || !_quests
                  [questId].IsActive) return;
54              if (amount < 0) throw new ArgumentException("Progress amount
                  must be non-negative.", nameof(amount));
55
56              var quest = _quests[questId];
57              quest.Progress = Math.Min(quest.Progress + amount,
                  quest.RequiredProgress);
58              quest.IsComplete = quest.Progress >= quest.RequiredProgress;
59              QuestUpdated?.Invoke(questId, quest.IsComplete);
60
61              if (quest.IsComplete)
62              {
63                  HandleQuestCompletion(questId);
64              }
65          }
66
67
68          /// Checks if a quest is complete.
69
70          public bool IsQuestComplete(string questId)
71          {
72              return _quests.ContainsKey(questId) && _quests
                  [questId].IsComplete;
73          }
74
75
76          /// Checks the favor result for a favor-based quest.
77
78          public Enum.FavorResult CheckFavorResult(string questId)
79          {
80              if (!_quests.ContainsKey(questId) || _quests[questId].Type !=
                  CustomProjectRPG.Entities.Enum.Enum.QuestType.Favor)
81              {
82                  return
                      CustomProjectRPG.Entities.Enum.Enum.FavorResult.Invalid;
83              }
```

```
84
85              var quest = _quests[questId];
86              if (quest.IsComplete)
87              {
88                  return
                      CustomProjectRPG.Entities.Enum.Enum.FavorResult.Success;
89              }
90              float favorThreshold = 50f + (_corruption.
                  CorruptionPercentage * 0.5f); // Adjust based on corruption
91              return quest.Progress >= favorThreshold ?
                  CustomProjectRPG.Entities.Enum.Enum.FavorResult.Success :
                  CustomProjectRPG.Entities.Enum.Enum.FavorResult.Insufficient
                  ;
92          }
93
94
95          /// Handles logic after quest completion (e.g., rewards, dialogue
              updates).
96
97          private void HandleQuestCompletion(string questId)
98          {
99              var quest = _quests[questId];
100             quest.IsActive = false; // Mark as inactive after completion
101                                     // Example: Trigger dialogue update
                                          via DialogueManager (assumes
                                          integration)
102                                     // dialogueManager.ShowDialogue
                                          (questId, "completed");
103         }
104
105
106         /// Determines the required progress based on quest type.
107
108         private int GetRequiredProgress
              (CustomProjectRPG.Entities.Enum.Enum.QuestType type)
109         {
110             return type switch
111             {
112                 CustomProjectRPG.Entities.Enum.Enum.QuestType.Daily => 5,
113                 CustomProjectRPG.Entities.Enum.Enum.QuestType.Story => 10,
114                 CustomProjectRPG.Entities.Enum.Enum.QuestType.Favor => 8,
115                 _ => 1
116             };
117         }
118
119
120         /// Serializes the quest state to JSON.
121
122         public string Serialize()
```

```
123         {
124             var data = new QuestManagerData
125             {
126                 Quests = _quests
127             };
128             return JsonSerializer.Serialize(data);
129         }
130 
131 
132         /// Deserializes the quest state from JSON.
133 
134         public void Deserialize(string json)
135         {
136             var data = JsonSerializer.Deserialize<QuestManagerData>(json)
137                 ?? throw new JsonException("Invalid JSON format for quest
                      state.");
138             _quests = data.Quests ?? new Dictionary<string, QuestData>();
139         }
140 
141         private class QuestData
142         {
143             [JsonPropertyName("type")]
144             public CustomProjectRPG.Entities.Enum.Enum.QuestType Type
                  { get; set; }
145 
146             [JsonPropertyName("isActive")]
147             public bool IsActive { get; set; }
148 
149             [JsonPropertyName("progress")]
150             public int Progress { get; set; }
151 
152             [JsonPropertyName("requiredProgress")]
153             public int RequiredProgress { get; set; }
154 
155             [JsonPropertyName("isComplete")]
156             public bool IsComplete { get; set; }
157         }
158 
159         private class QuestManagerData
160         {
161             [JsonPropertyName("quests")]
162             public Dictionary<string, QuestData> Quests { get; set; }
163         }
164     }
165 }
166 
```

```cs
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;

namespace CustomProjectRPG.Core.Utilities
{

    public static class SerializeJSON
    {
        public static string ToJson<T>(T data)
        {
            return JsonSerializer.Serialize(data);
        }

        public static T FromJson<T>(string json)
        {
            return JsonSerializer.Deserialize<T>(json);
        }
    }

}

```

```
1  using SplashKitSDK;
2  using System;
3  using System.Collections.Generic;
4  using System.Linq;
5  using System.Text;
6  using System.Threading.Tasks;
7  using CustomProjectRPG.Core.Utilities;
8  using CustomProjectRPG.Interfaces;
9  using Vector2D = CustomProjectRPG.Core.Utilities.Vector2D;
10
11 namespace CustomProjectRPG.UI
12 {
13     public class StaticAsset : ICollidable, IDraw
14     {
15         private readonly string _name;
16         private readonly string _description;
17         private readonly Hitbox _hitbox;
18         private readonly string _inspectionDialogue;
19
20         public string Name => _name;
21         public string Description => _description;
22         public Hitbox Hitbox => _hitbox;
23
24         public StaticAsset(string name, string description, Vector2D
             position, Vector2D size, string inspectionDialogue)
25         {
26             _name = name;
27             _description = description;
28             _hitbox = new Hitbox(position, size);
29             _inspectionDialogue = inspectionDialogue;
30         }
31
32         public void Draw()
33         {
34             // Placeholder: Draw static asset using SplashKit
35             SplashKit.FillRectangle(Color.Gray, _hitbox.Position.X,
                 _hitbox.Position.Y, _hitbox.Size.X, _hitbox.Size.Y);
36         }
37
38         public string Inspect()
39         {
40             // Return dialogue for inspection (triggered by player
                 pressing E)
41             return _inspectionDialogue;
42         }
43     }
44 }
45
```

```
 1  using CustomProjectRPG.Interfaces;
 2  using System;
 3  using System.Collections.Generic;
 4  using System.Linq;
 5  using System.Text;
 6  using System.Text.Json;
 7  using System.Threading.Tasks;
 8  using Enum = CustomProjectRPG.Entities.Enum.Enum;
 9  using System.Text.Json.Serialization;
10  using CustomProjectRPG.Core.Utilities;
11
12
13  namespace CustomProjectRPG.Entities.Component
14  {
15
16
17      public class StatsComponent : ISerializeJSON<StatsComponent>
18      {
19          private float _hp;
20          private float _maxHp;
21          private float _atk;
22          private float _def;
23          private float _agility;
24          private Enum.Enum.EntityType _entityType;
25
26          public float HP => _hp;
27          public float MaxHP => _maxHp;
28          public float ATK => _atk;
29          public float DEF => _def;
30          public float Agility => _agility;
31
32
33          /// Initializes a new StatsComponent with base stats for a
              specific entity type.
34
35          public StatsComponent(Enum.Enum.EntityType entityType, float
              initialHp = 100f, float initialAtk = 10f, float initialDef =
              5f, float initialAgility = 5f)
36          {
37              _entityType = entityType;
38              _maxHp = initialHp;
39              _hp = _maxHp;
40              _atk = initialAtk;
41              _def = initialDef;
42              _agility = initialAgility;
43          }
44
45
46          /// Applies damage to the entity's HP, ensuring it doesn't go
```

```
            below 0.
47
48          public void TakeDamage(float amount)
49          {
50              if (amount < 0) throw new ArgumentException("Damage must be
                  non-negative.", nameof(amount));
51              _hp = Math.Max(_hp - amount, 0f);
52          }
53
54
55          /// Heals the entity, capping HP at MaxHP.
56
57          public void Heal(float amount)
58          {
59              if (amount < 0) throw new ArgumentException("Heal amount must
                  be non-negative.", nameof(amount));
60              _hp = Math.Min(_hp + amount, _maxHp);
61          }
62
63
64          /// Applies corruption-based stat penalties using the provided
              CorruptionComponent.
65
66          public void ApplyCorruptionPenalty(ICorruption corruption)
67          {
68              if (corruption == null) throw new ArgumentNullException(nameof
                  (corruption));
69              if (_entityType != Enum.Enum.EntityType.Player) return;
70              float multiplier = corruption.GetStatMultiplier();
71              _atk *= multiplier;
72              _def *= multiplier;
73              _agility *= multiplier;
74          }
75
76
77          /// Resets stats to their base values (e.g., after a loop reset).
78
79          public void ResetStats()
80          {
81              _hp = _maxHp;
82              _atk = 10f;
83              _def = 6f;
84              _agility = 5f;
85          }
86
87
88          /// Serializes the stats state to JSON.
89
90          public string Serialize()
```

```
91          {
92              var data = new StatsData
93              {
94                  HP = _hp,
95                  MaxHP = _maxHp,
96                  ATK = _atk,
97                  DEF = _def,
98                  Agility = _agility
99              };
100             return SerializeJSON.ToJson(data);
101         }
102
103
104         /// Deserializes the stats state from JSON.
105
106         public void Deserialize(string json)
107         {
108             var data = SerializeJSON.FromJson<StatsData>(json);
109             _hp = Math.Clamp(data.HP, 0f, data.MaxHP);
110             _maxHp = data.MaxHP;
111             _atk = data.ATK;
112             _def = data.DEF;
113             _agility = data.Agility;
114         }
115
116         private class StatsData
117         {
118             [JsonPropertyName("hp")]
119             public float HP { get; set; }
120
121             [JsonPropertyName("maxHp")]
122             public float MaxHP { get; set; }
123
124             [JsonPropertyName("atk")]
125             public float ATK { get; set; }
126
127             [JsonPropertyName("def")]
128             public float DEF { get; set; }
129
130             [JsonPropertyName("agility")]
131             public float Agility { get; set; }
132         }
133     }
134
135 }
136
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace CustomProjectRPG.Core.Utilities
8  {
9
10     public struct Vector2D
11     {
12         public float X { get; set; }
13         public float Y { get; set; }
14
15         public Vector2D(float x, float y)
16         {
17             X = x;
18             Y = y;
19         }
20
21         public Vector2D Add(Vector2D other) => new Vector2D(X + other.X, Y
             + other.Y);
22         public Vector2D Scale(float factor) => new Vector2D(X * factor,
             Y * factor);
23         public float DistanceTo(Vector2D other) =>
24             (float)Math.Sqrt(Math.Pow(X - other.X, 2) + Math.Pow(Y -
                 other.Y, 2));
25     }
26 }
```